C# / .NET Learning Roadmap

Topic Notes — Section 4: Collections

Topics: 4.1 through 4.6

Date: 29 June 2026

Section 4 — Collections

This section covers the core collection types in .NET and the interfaces that unify them.

4.1. List

Builds on: 3.1 (generics), 1.8 (loops), 1.9 (arrays — List is a resizable array internally), 2.9 (interfaces — List implements IList, IEnumerable).

What is List and why does it exist?

Arrays are fixed-size — once created, you can't add or remove elements. List is a resizable, ordered collection that wraps an internal array and grows automatically when needed.

```
var list = new List<int> { 1, 2, 3 };
list.Add(4);        // grows automatically
list.Remove(2);     // removes elements
list.Insert(0, 0);  // inserts at any position
```

Creating a List

```
var names = new List<string>();                          // empty
var names = new List<string>(100);                       // with capacity hint
var names = new List<string> { "Sathish", "Priya" };     // with items
var names = existingCollection.ToList();                 // from IEnumerable
List<string> names = ["Sathish", "Priya", "Karthik"];  // C# 12
```

Core operations

```
// Adding:
orders.Add(new Order { Id = 1 });
orders.AddRange(otherOrders);
orders.Insert(0, new Order { Id = 0 });

// Removing:
orders.Remove(specificOrder);
orders.RemoveAt(0);
orders.RemoveAll(o => o.Amount < 0);
orders.Clear();

// Finding:
var order = orders.Find(o => o.Id == 42);
int index  = orders.FindIndex(o => o.Id == 42);
bool has   = orders.Contains(specificOrder);

// Sorting:
names.Sort();
orders.Sort((a, b) => a.Amount.CompareTo(b.Amount));
orders.Reverse();

// Reading:
int count  = orders.Count;   // NOT Length -- that's arrays
var first  = orders[0];
var last   = orders[^1];
```

Safe removal during iteration

```
// Iterate backwards:
for (int i = orders.Count - 1; i >= 0; i--)
    if (orders[i].IsCancelled) orders.RemoveAt(i);

// RemoveAll -- cleanest:
orders.RemoveAll(o => o.IsCancelled);
```

List internals — capacity vs count

```
// Count -- how many items; Capacity -- internal array size
var list = new List<int>(10000);   // pre-allocate to avoid resizing
list.TrimExcess();                  // reclaim excess capacity
```

Interface hierarchy

```
IList<Order> orders = new List<Order>();           // full read/write + indexer
ICollection<Order> orders = new List<Order>();     // Count, Add, Remove
IEnumerable<Order> orders = new List<Order>();     // iteration only
IReadOnlyList<Order> orders = new List<Order>();   // read + index, no mutation

// Expose read-only from public methods:
public IReadOnlyList<Order> GetOrders() => _orders.AsReadOnly();
```

Key Takeaways

- List is the default collection for ordered, mutable data in .NET
- Backed by an array that doubles in size when full — Count is items, Capacity is internal array size
- Don't modify a list while iterating with foreach — use RemoveAll or iterate backwards
- Expose as the narrowest interface: IReadOnlyList for read-only, IEnumerable for iteration-only

```
            | List<T>              | Array (T[])
------------+----------------------+------------------
Size        | Resizable            | Fixed
Add/Remove  | Yes                  | No
Property    | Count                | Length
```

.NET-specific rules:

- ToList() materialises a LINQ query — call it last
- ToListAsync() for EF Core queries — never call ToList() on an IQueryable
- Never expose a List field as a public property — use IReadOnlyList with AsReadOnly()

Memory aid: List is a stretchy array. Like a rubber band — it holds things in order and stretches to fit more.

Debugging Tips

Symptom	Cause	Fix
InvalidOperationException during foreach	Modifying list while iterating	Use RemoveAll, iterate backwards, or collect changes first
ArgumentOutOfRangeException on index access	Accessing index >= Count	Check Count before accessing by index
List returned from method can be mutated by caller	Returning List directly from public method	Return IReadOnlyList via AsReadOnly()
ToList() on EF Core query causes N+1	Called before Include or Where — loads everything then filters	Chain all LINQ operations before calling ToListAsync()
Count is 0 but list was populated	List was cleared, or you're looking at a different instance	Check if Clear() was called or if DI lifetime is wrong

Self-check — you should now be able to:

- Create a List and add, remove, insert, and find items
- Explain the difference between Count and Capacity
- Choose the right interface for method signatures

4.2. Dictionary

Builds on: 3.1 (generics — Dictionary has two type parameters), 2.17 (object equality — GetHashCode drives dictionary lookup), 4.1.

What is Dictionary and why does it exist?

A List lets you find items by position. Dictionary lets you find items by a meaningful key — O(1) vs O(n) for a list scan.

```
// Finding by value in a List -- O(n):
var order = orders.FirstOrDefault(o => o.Id == 42);

// Finding by key in a Dictionary -- O(1):
var order = orderDict[42];
```

Creating and adding

```
var statusMessages = new Dictionary<string, string>
{
    ["Pending"]  = "Your order is being reviewed",
    ["Shipped"]  = "Your order is on its way",
    ["Delivered"]= "Your order has been delivered"
};

var orderById = orders.ToDictionary(o => o.Id);

// Add -- throws if key already exists:
dict.Add(1, order1);

// Indexer -- adds or updates (never throws):
dict[1] = order1;
```

```
// TryAdd -- adds only if new, returns false otherwise:
bool added = dict.TryAdd(1, order1);
```

Safe reading with TryGetValue

```
// ALWAYS prefer TryGetValue over ContainsKey + indexer (one lookup vs two):
if (dict.TryGetValue(42, out Order? order))
    Console.WriteLine(order.CustomerName);

// GetValueOrDefault -- returns default if not found:
Order? order = dict.GetValueOrDefault(42);
```

Grouping and counting patterns

```
// Group orders by status using LINQ:
var ordersByStatus = orders
    .GroupBy(o => o.Status)
    .ToDictionary(g => g.Key, g => g.ToList());

// Count occurrences:
var counts = new Dictionary<string, int>();
foreach (var word in words)
    counts[word] = counts.GetValueOrDefault(word) + 1;
```

Custom key types and case-insensitive keys

```
// Custom key -- must have correct Equals/GetHashCode:
public record OrderKey(int CustomerId, string Currency);
var lookup = new Dictionary<OrderKey, List<Order>>();

// Case-insensitive string keys:
var dict = new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase);
dict["USD"] = "Dollar";
dict["usd"];   // found
```

Key Takeaways

- Dictionary provides O(1) key-based lookup
- TryGetValue is the safe, performant way to read — never ContainsKey + indexer
- dict[key] = value adds or replaces; dict.Add() throws on duplicate key
- Iteration order is not guaranteed — use SortedDictionary if order matters
- Custom key types must correctly implement Equals and GetHashCode — use records

```
Operation         | Method                    | Throws?
------------------+---------------------------+--------
Add (must be new) | dict.Add(key, val)        | Yes if key exists
Add or update     | dict[key] = val           | Never
Add if new only   | dict.TryAdd(key, val)     | Never -- returns bool
Get (must exist)  | dict[key]                 | Yes if key missing
Get safely        | dict.TryGetValue(k, out v)| Never -- returns bool
Get or default    | dict.GetValueOrDefault(k) | Never
```

Memory aid: Dictionary is a phone book. You look up by name (key) and instantly find the number (value).

Debugging Tips

Symptom	Cause	Fix
KeyNotFoundException	Key not in dictionary, accessed with dict[key]	Use TryGetValue or GetValueOrDefault
ArgumentException: key already exists	dict.Add() called with existing key	Use dict[key] = value to add or replace
Custom key not found despite same values	Key type missing Equals/GetHashCode	Use a record or implement equality correctly
ToDictionary throws on duplicate keys	Source collection has duplicate key values	Use GroupBy then ToDictionary
Case-sensitive key mismatch	USD vs usd treated as different keys	Pass StringComparer.OrdinalIgnoreCase to constructor
Race condition on shared dictionary	Dictionary is not thread-safe	Use ConcurrentDictionary or proper locking

Self-check — you should now be able to:

- Explain the difference between dict.Add(), dict[key] = value, and dict.TryAdd()
- Use TryGetValue instead of ContainsKey + indexer and explain why
- Group a flat list into a Dictionary> using LINQ

4.3. HashSet, SortedSet

Builds on: 4.2 (Dictionary — HashSet uses the same hash table mechanism), 2.17 (object equality), 4.1 (List — sets solve the uniqueness problem).

What is a HashSet and why does it exist?

HashSet is an unordered collection where every item is unique, membership testing is O(1), and set operations (union, intersection, difference) are built in.

```
var set = new HashSet<string> { "A", "B", "A", "C" };
// Contains: A, B, C -- duplicate A silently ignored

bool added = set.Add("A");   // false -- already exists, no exception
bool added2 = set.Add("D");  // true -- new item

set.Contains("A");   // true -- O(1)
set.Remove("A");     // removes if present

// Case-insensitive:
var set = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
set.Add("USD"); set.Add("usd");
Console.WriteLine(set.Count);   // 1
```

Set operations — the unique power of HashSet

```
var setA = new HashSet<int> { 1, 2, 3, 4, 5 };
var setB = new HashSet<int> { 3, 4, 5, 6, 7 };

// All operations modify setA in place -- clone first to preserve:
var result = new HashSet<int>(setA);   // clone
result.IntersectWith(setB);            // { 3, 4, 5 }

// UnionWith     -- all items from both
// IntersectWith -- only items in both
// ExceptWith    -- items in setA but not setB
// SymmetricExceptWith -- items in either but not both

// Non-mutating checks:
setA.IsSubsetOf(setB)   // all of setA is in setB?
setA.Overlaps(setB)     // any items in common?
setA.SetEquals(setB)    // same items?
```

Deduplication and the List.Contains performance fix

```
// Deduplication:
var uniqueIds = new HashSet<int>(rawIds);

// Performance bug -- O(n squared):
var processed = new List<int>();
foreach (var id in orderIds)
    if (!processed.Contains(id)) { ProcessOrder(id); processed.Add(id); }

// Fix -- O(n):
var processed = new HashSet<int>();
foreach (var id in orderIds)
    if (processed.Add(id)) ProcessOrder(id);   // Add returns false if duplicate

// Converting lookup List to HashSet before LINQ Where:
var premiumIds = new HashSet<int>(premiumUserIdList);
var premiumOrders = orders.Where(o => premiumIds.Contains(o.UserId));
```

SortedSet

```
var sorted = new SortedSet<int> { 5, 3, 1, 4, 2 };
foreach (var n in sorted) Console.Write($"{n} ");   // 1 2 3 4 5 -- always sorted

sorted.Min;   // 1 -- O(log n)
sorted.Max;   // 5 -- O(log n)

var range = sorted.GetViewBetween(2, 4);   // { 2, 3, 4 }

// Custom sort order:
var set = new SortedSet<string>(Comparer<string>.Create((a, b) =>
    a.Length != b.Length ? a.Length.CompareTo(b.Length) : string.Compare(a, b)));
```

Key Takeaways

- HashSet = unordered, unique items, O(1) add/remove/contains
- Duplicate Add returns false but doesn't throw — use this for 'process only if new' patterns

- Set operations (UnionWith, IntersectWith, ExceptWith) modify in place — clone first
- SortedSet = sorted unique items, O(log n)
- Convert lookup lists to HashSet before use in LINQ Where — major performance win

```
Need                                      | Use
------------------------------------------+------------------
Unique items, fast lookup, no order       | HashSet<T>
Unique items, always sorted               | SortedSet<T>
Set operations (union, intersect, etc.)   | HashSet<T>
Fast membership check in LINQ filter      | HashSet<T>
```

Memory aid: HashSet is a bouncer at a club. The rule: no duplicates allowed. Checks in O(1) using the hash — no scanning every guest.

Debugging Tips

Symptom	Cause	Fix
Duplicates still appearing in HashSet	Custom type missing Equals/GetHashCode	Use record or implement equality correctly
HashSet.Contains returning wrong result	Same cause — wrong hash/equality	Fix GetHashCode to be consistent with Equals
Set operation mutated original	UnionWith/IntersectWith modify in place	Clone with new HashSet(original) before operating
SortedSet ordering unexpected	Custom IComparer returning wrong values	Debug the comparer — return negative/zero/positive correctly
List.Contains slow in loop	O(n) per call on a list	Convert list to HashSet before the loop

Self-check — you should now be able to:

- Explain why HashSet is faster than List for membership testing
- Apply UnionWith, IntersectWith, ExceptWith and explain they modify in place
- Convert a lookup List to HashSet before a LINQ Where clause

4.4. Queue, Stack

Builds on: 4.1 (List — Queue and Stack are specialised with restricted access), 3.1 (generics), 1.8 (loops).

What are Queue and Stack and why do they exist?

Queue — First In, First Out (FIFO). Like a queue at a ticket counter. Stack — Last In, First Out (LIFO). Like a stack of plates.

```
// Queue -- FIFO:
var queue = new Queue<string>();
queue.Enqueue("First"); queue.Enqueue("Second"); queue.Enqueue("Third");

string first = queue.Dequeue();   // "First"
string next  = queue.Peek();      // "Second" -- still in queue
if (queue.TryDequeue(out string? item)) Console.WriteLine(item);
```

```
if (queue.TryPeek(out string? next2))  Console.WriteLine(next2);

// Stack -- LIFO:
var stack = new Stack<string>();
stack.Push("First"); stack.Push("Second"); stack.Push("Third");

string top = stack.Pop();   // "Third" -- last added, first out
string peek = stack.Peek(); // "Second" -- still on stack

if (stack.TryPop(out string? popped)) Console.WriteLine(popped);

// foreach iterates WITHOUT consuming items -- items remain after loop
```

Undo/redo with Stack

```
public class TextEditor
{
    private string _content = string.Empty;
    private readonly Stack<string> _undoStack = new();
    private readonly Stack<string> _redoStack = new();

    public void Type(string text)
    { _undoStack.Push(_content); _redoStack.Clear(); _content += text; }

    public void Undo()
    {
        if (_undoStack.TryPop(out var previous))
        { _redoStack.Push(_content); _content = previous; }
    }

    public void Redo()
    {
        if (_redoStack.TryPop(out var next))
        { _undoStack.Push(_content); _content = next; }
    }
}
```

Thread-safe alternatives

```
// ConcurrentQueue<T> -- thread-safe FIFO:
private readonly ConcurrentQueue<WorkItem> _workQueue = new();
_workQueue.Enqueue(new WorkItem { Id = 1 });
if (_workQueue.TryDequeue(out var item)) await ProcessAsync(item);

// Channel<T> -- modern async producer/consumer (preferred in ASP.NET Core):
var channel = Channel.CreateUnbounded<EmailMessage>();
builder.Services.AddSingleton(channel.Writer);
builder.Services.AddSingleton(channel.Reader);

// Producer: await emailWriter.WriteAsync(new EmailMessage(...));
// Consumer BackgroundService:
await foreach (var message in reader.ReadAllAsync(ct))
    await SendEmailAsync(message);
```

Key Takeaways

- Queue = FIFO — Enqueue adds to back, Dequeue removes from front
- Stack = LIFO — Push adds to top, Pop removes from top
- Both have Peek and Try variants to avoid exceptions on empty
- Neither is thread-safe — use ConcurrentQueue or Channel for concurrent scenarios
- Channel is the modern async-native producer/consumer for ASP.NET Core

```
            | Queue<T>           | Stack<T>
------------+--------------------+-----------------
Pattern     | FIFO               | LIFO
Add         | Enqueue            | Push
Remove      | Dequeue            | Pop
Real use    | Job queues, BFS    | Undo, DFS, parsing
Thread-safe | No                 | No
TS version  | ConcurrentQueue    | ConcurrentStack
```

Memory aid: Queue = post office queue. First person in line gets served first. Stack = stack of dirty dishes. Last plate added is the first one washed.

Debugging Tips

Symptom	Cause	Fix
InvalidOperationException on Dequeue/Pop	Called on empty collection	Use TryDequeue/TryPop and check return value
Items processed in wrong order	Used Stack when Queue was needed or vice versa	Verify FIFO vs LIFO requirement for your problem
Race condition in shared queue	Queue used across threads without synchronisation	Switch to ConcurrentQueue or Channel
foreach loop consumed the queue	Misconception — foreach does not consume items	Items remain after foreach; only Dequeue/Pop consume
Background job channel not processing	ChannelReader.ReadAllAsync not awaited properly	Ensure await foreach with CancellationToken in BackgroundService

Self-check — you should now be able to:

- Explain the difference between FIFO and LIFO with a real-world analogy
- Implement a basic undo/redo mechanism using Stack
- Choose between Queue, ConcurrentQueue, and Channel for a given scenario

4.5. IEnumerable, ICollection, IList Interfaces

Builds on: 2.9 (interfaces), 4.1 (List), 3.1 (generics), 1.8 (loops — foreach works on anything implementing IEnumerable).

What are these interfaces and why do they exist?

These interfaces form a hierarchy of capabilities from minimal (just iterate) to full (read, write, index). Using the narrowest interface that satisfies your needs makes your code more flexible and more honest about its requirements.

```
IEnumerable<T>                   -- iteration only (foreach, LINQ)
    +-- ICollection<T>           -- + Count, Add, Remove, Contains
            +-- IList<T>         -- + index access [i], Insert, RemoveAt

IEnumerable<T>                   -- read+write branch:
    +-- IReadOnlyCollection<T>   -- Count + iteration, no write
            +-- IReadOnlyList<T> -- Count + iteration + index, no write
```

IEnumerable — iteration only and deferred execution

```
// Works with anything implementing IEnumerable<T>:
public void PrintOrders(IEnumerable<Order> orders)
{
    foreach (var order in orders)
        Console.WriteLine($"{order.Id}: {order.Amount:C}");
}

PrintOrders(new List<Order>());
PrintOrders(new Order[] { });
PrintOrders(dbContext.Orders.Where(o => o.IsActive));   // EF Core

// DEFERRED EXECUTION -- the enumerable is a recipe, not a result:
IEnumerable<Order> query = orders.Where(o => o.Amount > 100);
// Nothing executed yet

foreach (var order in query) Console.WriteLine(order.Id);   // executes now
foreach (var order in query) Console.WriteLine(order.Id);   // executes AGAIN

// For EF Core -- materialise once:
List<Order> orders = context.Orders.Where(o => o.IsActive).ToList();
```

ICollection, IList, and read-only interfaces

```
// ICollection<T> -- adds Count, Add, Remove, Contains:
public void AddPendingOrders(ICollection<Order> dest, IEnumerable<Order> src)
{
    foreach (var order in src.Where(o => o.Status == "Pending"))
        dest.Add(order);
    Console.WriteLine($"Collection now has {dest.Count} items");
}

// IList<T> -- adds index access, Insert, RemoveAt:
public void Shuffle<T>(IList<T> list)
{
    var rng = Random.Shared;
    for (int i = list.Count - 1; i > 0; i--)
    {
        int j = rng.Next(i + 1);
        (list[i], list[j]) = (list[j], list[i]);
    }
}
```

```
// Read-only from public methods:
public IReadOnlyList<Order> GetAll() => _orders.AsReadOnly();
```

Decision guide

```
// Only iterates -> IEnumerable<T>:
public decimal CalculateTotal(IEnumerable<Order> orders) => orders.Sum(o => o.Amount);

// Needs Count without iterating -> IReadOnlyCollection<T>:
public bool IsEmpty(IReadOnlyCollection<Order> orders) => orders.Count == 0;

// Needs to add items -> ICollection<T>:
public void Populate(ICollection<Order> destination) => destination.Add(new Order());

// Needs index access -> IList<T>:
public Order GetLast(IList<Order> orders) => orders[orders.Count - 1];
```

IQueryable — EF Core

```
// IQueryable<T> -- pending SQL query, add filters before executing:
IQueryable<Order> query = context.Orders;
query = query.Where(o => o.UserId == userId);    // adds WHERE to SQL
query = query.OrderBy(o => o.CreatedAt);         // adds ORDER BY
List<Order> orders = await query.ToListAsync();  // executes once

// Never materialise early:
// BAD: context.Orders.ToList().Where(o => o.UserId == userId) -- loads ALL
// GOOD: context.Orders.Where(o => o.UserId == userId).ToList()
```

Key Takeaways

- IEnumerable — iteration only, deferred execution, the base of all collections
- ICollection — adds Count, Add, Remove, Contains
- IList — adds index access, Insert, RemoveAt
- IReadOnlyList / IReadOnlyCollection — read access without mutation
- Use the narrowest interface in method parameters — maximise flexibility
- Return narrower interfaces from public methods — prevent unintended mutation
- IQueryable is not a collection — it's a pending SQL query; don't materialise early

```
Interface              | Count | Add/Remove | Index [i] | Mutation
-----------------------+-------+------------+-----------+---------
IEnumerable<T>         | No    | No         | No        | No
IReadOnlyCollection<T> | Yes   | No         | No        | No
IReadOnlyList<T>       | Yes   | No         | Yes       | No
ICollection<T>         | Yes   | Yes        | No        | Yes
IList<T>               | Yes   | Yes        | Yes       | Yes
```

Memory aid: IEnumerable = a conveyor belt — watch items pass by. ICollection = a basket — count and add/remove. IList = a numbered shelf — access any item by position.

Debugging Tips

Symptom	Cause	Fix
EF Core query executed multiple times	IEnumerable re-evaluated each foreach — deferred execution	Call .ToList() or .ToListAsync() to materialise once
Count() on IEnumerable is slow	Iterates entire sequence each time	Use ICollection.Count property or materialise first
Can't call .Count (only .Count())	Variable is IEnumerable — no Count property	Change parameter type to ICollection or call .Count() accepting O(n)
Caller mutating returned collection	Returning List directly from public method	Return IReadOnlyList via .AsReadOnly()
Premature SQL execution	.ToList() called before all filters applied	Chain all LINQ operations before materialising

.NET-specific: IEnumerable from an EF Core context — if the DbContext is disposed before iteration, you get ObjectDisposedException. Always materialise EF Core results within the scope where the context is alive.

Self-check — you should now be able to:

- Explain the hierarchy from IEnumerable to ICollection to IList
- Explain deferred execution in IEnumerable and when to call .ToList()
- Explain why IQueryable should not be materialised early

4.6. IComparable / IComparer for Custom Sorting

Builds on: 4.1 (List — Sort() uses these interfaces), 4.3 (SortedSet — uses IComparer), 2.9 (interfaces), 2.17 (object equality — sorting is the ordering counterpart).

What are these interfaces and why do they exist?

Sorting requires answering: 'which of these two items comes first?' IComparable bakes one natural ordering into the type itself. IComparer is an external, pluggable ordering — enabling multiple sort orders for the same type.

IComparable — natural ordering

CompareTo must return: Negative (this comes before other), Zero (equivalent), Positive (this comes after other).

```
public class Order : IComparable<Order>
{
    public int Id { get; }
    public decimal Amount { get; }
    public DateTime CreatedAt { get; }

    public Order(int id, decimal amount, DateTime createdAt)
        => (Id, Amount, CreatedAt) = (id, amount, createdAt);

    // Natural order -- by CreatedAt ascending:
    public int CompareTo(Order? other)
```

```
    {
        if (other is null) return 1;
        return CreatedAt.CompareTo(other.CreatedAt);
    }
}

orders.Sort();   // uses IComparable<Order>.CompareTo
var sorted = new SortedSet<Order>(orders);   // also uses IComparable<Order>
```

Multi-field ordering

```
public int CompareTo(Employee? other)
{
    if (other is null) return 1;

    int deptComp = string.Compare(Department, other.Department, StringComparison.Ordinal);
    if (deptComp != 0) return deptComp;

    int nameComp = string.Compare(LastName, other.LastName, StringComparison.Ordinal);
    if (nameComp != 0) return nameComp;

    return other.Salary.CompareTo(Salary);   // descending (reversed)
}
```

IComparer and Comparer.Create

```
// Full class:
public class OrderByAmountAsc : IComparer<Order>
{
    public int Compare(Order? x, Order? y)
    {
        if (x is null && y is null) return 0;
        if (x is null) return -1; if (y is null) return 1;
        return x.Amount.CompareTo(y.Amount);
    }
}

// Inline with Comparer<T>.Create (no class needed):
var byAmount = Comparer<Order>.Create((x, y) => x.Amount.CompareTo(y.Amount));
var byAmtDesc = Comparer<Order>.Create((x, y) => y.Amount.CompareTo(x.Amount));

orders.Sort(byAmount);
var sortedSet = new SortedSet<Order>(byAmtDesc);
```

List.Sort() overloads and LINQ ordering

```
// Sort in place -- Comparison<T> lambda:
orders.Sort((a, b) => a.Amount.CompareTo(b.Amount));

// LINQ -- creates new sequence, original unchanged, stable sort:
var sorted = orders.OrderBy(o => o.Amount).ToList();
var sorted2 = orders
    .OrderBy(o => o.CustomerName)
    .ThenByDescending(o => o.Amount)
    .ToList();
```

```
// EF Core -- always use LINQ (translates to SQL ORDER BY):
var orders = await context.Orders
    .OrderBy(o => o.CreatedAt)
    .ThenByDescending(o => o.Amount)
    .ToListAsync();
```

StringComparer

```
// Case-insensitive sorted collections:
var dict = new SortedDictionary<string, Order>(StringComparer.OrdinalIgnoreCase);
var set  = new SortedSet<string>(StringComparer.OrdinalIgnoreCase);

// Available comparers:
StringComparer.Ordinal              // byte-by-byte, fast, case-sensitive
StringComparer.OrdinalIgnoreCase    // byte-by-byte, fast, case-insensitive
StringComparer.CurrentCulture       // locale-aware, case-sensitive
StringComparer.InvariantCulture     // invariant locale, case-sensitive
```

Key Takeaways

- IComparable — implement on your type for one natural sort order
- IComparer — separate object; enables multiple sort orders for the same type
- CompareTo/Compare: negative = comes before, zero = equivalent, positive = comes after
- Comparer.Create(lambda) — create an IComparer inline without writing a class
- LINQ OrderBy/ThenBy — cleaner multi-key sorting, translates to SQL for EF Core
- StringComparer.OrdinalIgnoreCase — use for string keys in sorted collections

```
                  | IComparable<T>      | IComparer<T>
------------------+---------------------+----------------------
Implemented by    | The type itself     | A separate class
Number of orders  | One (natural)       | Unlimited
Creation          | Implement interface | Class or Create()
Best for          | One obvious order   | Multiple orderings
```

.NET-specific rules:

- List.Sort() — in-place, unstable sort
- LINQ OrderBy — stable sort (equal elements preserve original order)
- For EF Core — always use LINQ OrderBy, never List.Sort() on the query

Memory aid: IComparable is like a person who knows their own place in line. IComparer is like a referee who stands outside and decides who goes first.

Debugging Tips

Symptom	Cause	Fix
InvalidOperationException on Sort()	Type doesn't implement IComparable	Implement IComparable or pass a Comparison/IComparer
Sort order is inconsistent	CompareTo not transitive or consistent	Review comparison logic — if a<b and b<c, then a<c must hold
String sort order unexpected	Default culture-sensitive comparison used	Use StringComparer.Ordinal or StringComparison.Ordinal explicitly

Symptom	Cause	Fix
EF Core ignoring OrderBy	OrderBy applied after ToList() — sorts in memory not SQL	Apply OrderBy before ToListAsync()
SortedSet not deduplicating	IComparer.Compare returns non-zero for equal items	Ensure comparer returns 0 for logically equal items

Practice Problem

You have a list of Product objects with Name, Category, Price, and StockLevel properties:

- Implement IComparable with natural ordering by Name alphabetically (case-insensitive)
- Create an IComparer that sorts by Category ascending, then Price descending within the same category
- Sort a list using the natural order, then sort with the comparer
- Create a SortedSet using the comparer — verify three same-category products are sorted correctly

Self-check — you should now be able to:

- Implement IComparable on a class and explain what the return values mean
- Write an IComparer class for an alternative sort order
- Use Comparer.Create() to avoid writing a full class for simple comparisons
- Explain the difference between List.Sort() and LINQ OrderBy

---

Section 4 complete. All 6 topics from 4.1 through 4.6 are covered. Say next to begin Section 5: LINQ.